

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 3

Study and Empirical Analysis of Algorithms:

Depth First Search (DFS) and Breadth First Search (BFS)

Elaborated:

st. gr. FAF-242

Bogdan Arama

Verified:

asist. univ.

Fiștic Cristofor

Summary

ALGORITHM ANALYSIS	3
Objective	3
Tasks	3
Theoretical Notes	3
Introduction	4
Comparison Metric	4
Input Format	4
GRAPH TRAVERSAL ALGORITHMS	5
Breadth First Search (BFS)	5
Depth First Search (DFS)	6
EMPIRICAL RESULTS	7
Execution Time Comparison	7
Execution Time by Graph Family	8
Operations Count Comparison	9
CONCLUSION	10

ALGORITHM ANALYSIS

Objective

The objective of this laboratory work is to implement Breadth First Search (BFS) and Depth First Search (DFS), establish the properties of the input graphs used for testing, choose comparison metrics, perform empirical analysis, and interpret the results through graphical visualizations.

Tasks

1. Implement BFS and DFS in a programming language;
2. Establish the properties of the input data used for analysis;
3. Choose metrics for comparing the algorithms;
4. Perform empirical analysis of the proposed algorithms;
5. Present the obtained data graphically;
6. Deduce conclusions based on the experimental results.

Theoretical Notes

Graph traversal algorithms systematically visit vertices and explore edges. BFS explores vertices in layers using a queue, which makes it suitable for finding shortest paths in unweighted graphs. DFS explores as far as possible along a branch before backtracking, using a stack (explicitly or through recursion).

Both algorithms have theoretical time complexity $O(V + E)$ for an adjacency list representation, where V is the number of vertices and E is the number of edges. Empirical analysis complements this theory by measuring real execution time and the number of neighbor checks on concrete graph families.

The general procedure followed in this study involves establishing a goal, choosing a metric for efficiency, defining input characteristics, implementing the algorithms in Python, generating multiple graph instances, running the program for each input, and analyzing the obtained data.

Introduction

Graphs model relationships between entities: social networks, maps, dependency trees, and state spaces. BFS and DFS are fundamental building blocks for connectivity checks, cycle detection, topological sorting, and pathfinding.

Although both traversals visit the same vertices in a connected component when starting from the same node, they differ in visit order. BFS visits nodes by increasing distance from the start; DFS goes deep before visiting siblings. This laboratory compares their practical performance on sparse, dense, tree, and grid graphs.

Comparison Metric

The primary comparison metric is execution time $T(V)$, measured in seconds using Python's `perf_counter`. Each measurement is repeated three times and the minimum value is kept to reduce timer noise.

A secondary metric is the number of **neighbor checks** (operations), counting how many times an algorithm inspects an adjacent vertex. This metric reflects structural work and depends mainly on graph density, not on queue versus stack overhead.

Input Format

Graphs are represented as **adjacency lists**. All generated graphs are undirected. Traversals start from vertex 0. Vertex sizes used in benchmarks:

100, 250, 500, 750, 1000, 1500, 2000, 3000.

Four graph families were generated:

- **Sparse:** $E \approx V$ (few edges, average degree about 2);
- **Dense:** $E \approx V \cdot \log_2 V$ (many random edges);
- **Tree:** $E = V - 1$, connected, acyclic random tree;
- **Grid:** nodes on a square lattice with right/down links (actual V may be rounded to a perfect square).

Additional manual JSON inputs (`sample_small.json`, `sample_disconnected.json`) demonstrate traversal order on small graphs and behavior on disconnected components.

GRAPH TRAVERSAL ALGORITHMS

1. Breadth First Search (BFS)

Overview

BFS uses a FIFO queue. The start vertex is enqueued first. When a vertex is dequeued, all unvisited neighbors are enqueued. This guarantees level-by-layer exploration and is the standard choice for shortest paths in unweighted graphs.

How It Works

The algorithm marks the start vertex as visited and places it in a queue. While the queue is not empty, it removes the front vertex, records it in the visit order, and enqueues every neighbor that has not been visited yet. Because vertices are processed in arrival order, all nodes at distance k are visited before any node at distance $k + 1$.

Algorithm 1 BFS(G , start)

```
1: mark start as visited
2: enqueue start
3: while queue is not empty do
4:    $u \leftarrow$  dequeue
5:   for each neighbor  $v$  of  $u$  do
6:     if  $v$  is not visited then
7:       mark  $v$  as visited
8:       enqueue  $v$ 
9:     end if
10:  end for
11: end while
```

Listing 1: BFS implementation (algorithms/bfs.py)

```
1 from collections import deque
2
3
4 def bfs(adjacency_list, start=0):
5     """
6     Breadth-first search on an adjacency list.
7     Returns (visit_order, visited_count, operations_count).
8     operations_count counts neighbor checks (useful for empirical
9     comparison).
10    """
```

```

10     n = len(adjacency_list)
11     if n == 0:
12         return [], 0, 0
13
14     start = start % n
15     visited = [False] * n
16     order = []
17     operations = 0
18
19     queue = deque([start])
20     visited[start] = True
21
22     while queue:
23         node = queue.popleft()
24         order.append(node)
25
26         for neighbor in adjacency_list[node]:
27             operations += 1
28             if not visited[neighbor]:
29                 visited[neighbor] = True
30                 queue.append(neighbor)
31
32     return order, sum(visited), operations

```

Complexity

- **Time Complexity:** $O(V + E)$ with adjacency list representation.
- **Space Complexity:** $O(V)$ for the visited array and queue.

2. Depth First Search (DFS)

Overview

DFS uses a LIFO stack (iterative implementation in this laboratory). The start vertex is pushed first. When a vertex is popped, unvisited neighbors are pushed onto the stack. DFS is widely used for cycle detection, topological sorting, and exploring deep structure in a graph.

How It Works

The algorithm marks the start vertex and pushes it onto a stack. While the stack is not empty, it pops a vertex, records it, and pushes all unvisited neighbors. Neighbors are

processed in reverse order so that traversal order remains deterministic and comparable across runs.

Algorithm 2 DFS(G , start) — iterative

```
1: mark start as visited
2: push start onto stack
3: while stack is not empty do
4:    $u \leftarrow \text{pop}$ 
5:   for each neighbor  $v$  of  $u$  in reverse order do
6:     if  $v$  is not visited then
7:       mark  $v$  as visited
8:       push  $v$ 
9:     end if
10:  end for
11: end while
```

Listing 2: DFS implementation (algorithms/dfs.py)

```
1 def dfs(adjacency_list, start=0):
2     """
3     Depth-first search (iterative, stack-based) on an adjacency
4     list.
5     Returns (visit_order, visited_count, operations_count).
6     """
7     n = len(adjacency_list)
8     if n == 0:
9         return [], 0, 0
10
11     start = start % n
12     visited = [False] * n
13     order = []
14     operations = 0
15
16     stack = [start]
17     visited[start] = True
18
19     while stack:
20         node = stack.pop()
21         order.append(node)
22
23         # Reverse so smaller-index neighbors are visited first (
24         # stable DFS order).
```

```
23         for neighbor in reversed(adjacency_list[node]):
24             operations += 1
25             if not visited[neighbor]:
26                 visited[neighbor] = True
27                 stack.append(neighbor)
28
29     return order, sum(visited), operations
```

Complexity

- **Time Complexity:** $O(V + E)$ with adjacency list representation.
- **Space Complexity:** $O(V)$ for the visited array and stack.

EMPIRICAL RESULTS

Benchmarks were executed with `main.py`. Table ?? summarizes selected results at the largest tested size ($V \approx 3000$). Grid graphs use $V = 3025$ because the generator rounds to a square lattice.

Table 1: Selected benchmark results at large V .

Type	V	E	BFS (s)	DFS (s)	Ops
sparse	3000	3000	0.000531	0.000667	5758
dense	3000	34652	0.002939	0.003361	69304
tree	3000	2999	0.000575	0.000762	5998
grid	3025	5940	0.000775	0.000952	11880

Execution Time Comparison

Figure ?? aggregates all BFS and DFS timings on one chart. Dense graphs dominate runtime because they contain far more edges. Sparse, tree, and grid families remain near the bottom of the scale. For the same graph type, DFS is slightly slower than BFS due to stack operations and Python-level overhead, while both remain consistent with $O(V + E)$.

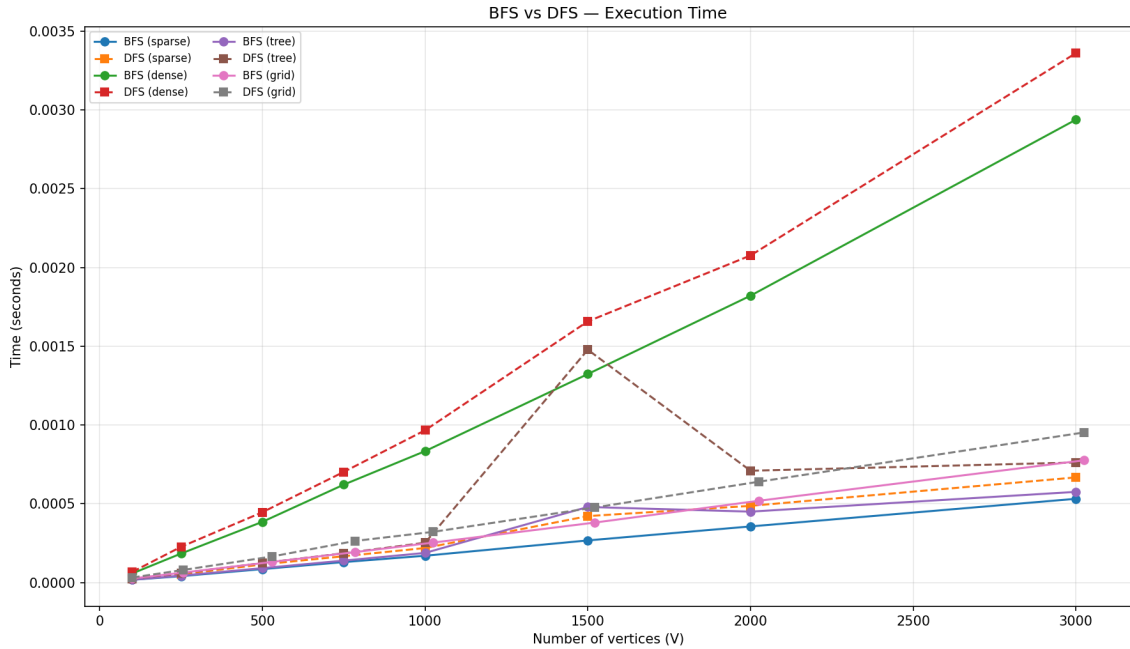


Figure 1: BFS vs DFS — execution time (all graph families).

Execution Time by Graph Family

Figure ?? splits the comparison into four panels so trends are visible for sparse and tree graphs, which are compressed in the combined plot. Dense graphs show the steepest growth with V . The tree DFS curve shows a small spike at $V = 1500$ caused by measurement noise rather than a fundamental algorithmic difference.

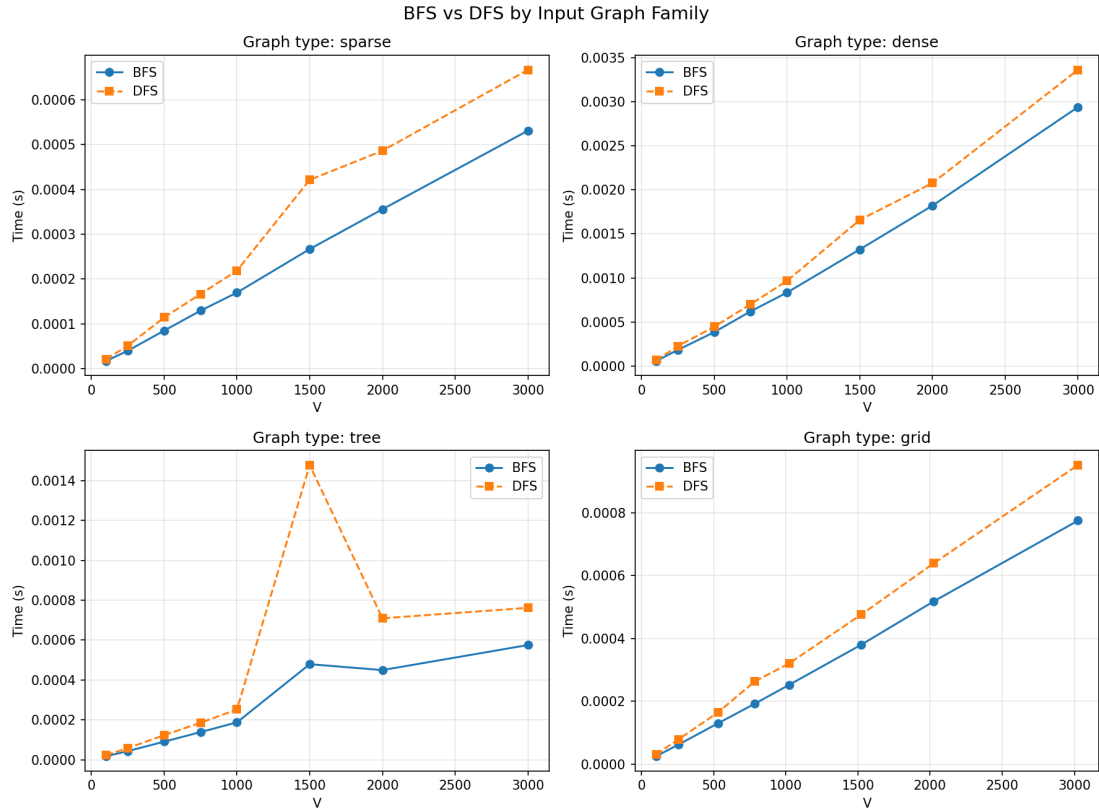


Figure 2: BFS vs DFS execution time by graph family.

Operations Count Comparison

Figure ?? plots neighbor checks instead of seconds. For each graph family, BFS and DFS curves overlap, confirming that both algorithms perform the same structural work on the same graph. Dense graphs require the most checks ($\approx 69\,000$ at $V = 3000$), while sparse and tree graphs need the fewest ($\approx 6\,000$). Grid graphs sit between them.

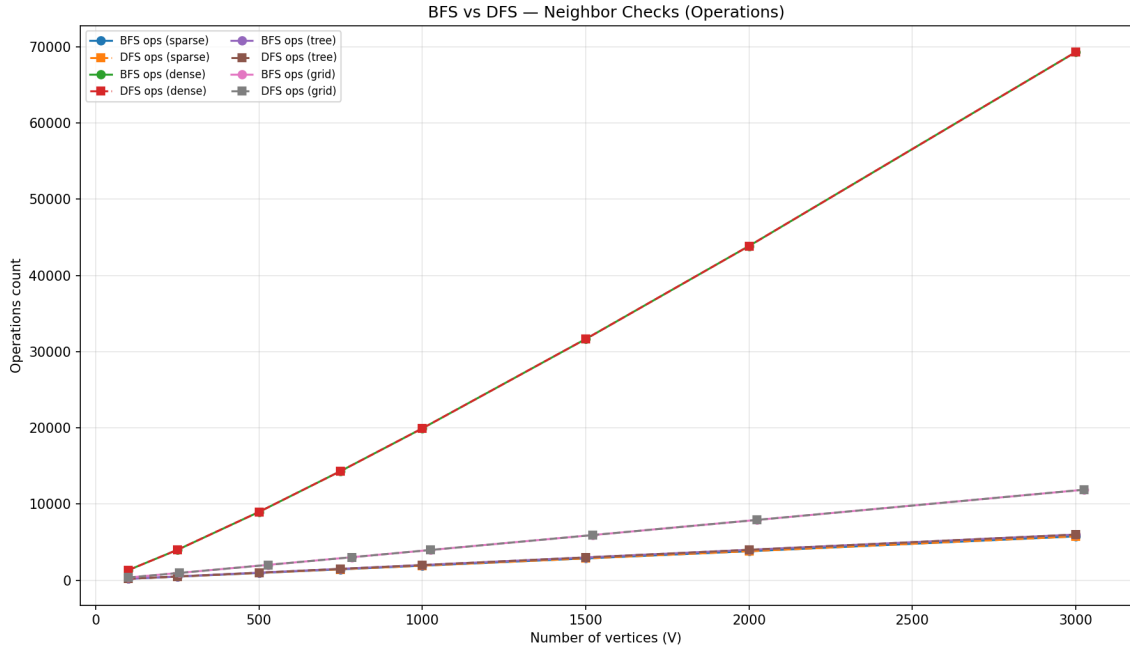


Figure 3: BFS vs DFS — neighbor checks (operations).

Small Graph Traversal Orders

On `sample_small.json` (6 connected vertices), BFS order: [0, 1, 2, 3, 4, 5]. DFS order: [0, 1, 3, 4, 5, 2]. Both visit all vertices, but in different order.

On `sample_disconnected.json`, starting at node 0 visits only component {0,1,2}; vertices 3–7 belong to other components and are never reached.

CONCLUSION

The empirical analysis compared Breadth First Search and Depth First Search on four graph families with increasing vertex counts. Both algorithms follow the theoretical $O(V + E)$ behavior on adjacency lists, but the constant factors depend strongly on the number of edges.

Dense graphs produced the highest execution times and the largest number of neighbor checks. At $V = 3000$, dense DFS required about 0.00336 seconds versus about 0.00067 seconds for sparse DFS, confirming that edge count is the dominant factor in practical runtime.

Sparse, tree, and grid graphs produced similar low runtimes because each vertex has a small bounded degree. BFS was consistently slightly faster than DFS on timed experiments, although the operations metric showed identical neighbor checks for both algorithms on the same input.

The identical operations curves demonstrate that choosing BFS or DFS does not change how many edges are inspected when both use the same visited marking strategy. The difference appears only in traversal order and minor implementation overhead (queue versus stack).

For unweighted shortest path problems or level-order processing, BFS is the appropriate choice. For deep structural exploration, cycle detection, or topological sorting pipelines, DFS is preferred. On very large sparse graphs both remain efficient; on dense graphs, memory traffic and neighbor iteration dominate and algorithm choice matters less than graph density.

REFERENCES

1. GeeksforGeeks — *Breadth First Search (BFS) for a Graph*.
Available at: [geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph](https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph)
2. GeeksforGeeks — *Depth First Search (DFS) for a Graph*.
Available at: [geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph](https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph)

Access the full implementation and analysis on GitHub:

<https://github.com/bogdan456tech/aa-labs/tree/main/lab3>